Erweitern und Festigen der erlernten Konzepte

Oktober 2025

Plan für die Woche

Montag

- Kleinere Themen abarbeiten

Dienstag

- Overload vs Override
- Besprechung PDF

Mittwoch

- Lernstandskontrolle
- Abstrakte Klassen

Donnerstag

- Einführung in Interfaces
- Wie fandet ihr die Woche?
- Update Selbsteinschätzung 811

Freitag

- Tag der Deutschen Einheit

Plan für heute

- Einführung Interfaces
- Vergleich abstrakte Klassen vs. Interfaces

Konzepte der OOP

- OOP steht für objektorientierte Programmierung

- Konzepte:
  - Kapselung (encapsulation)
  - Abstraktion (abstraction)
  - Vererbung (inheritance)
  - Polymorphismus (polymorphism)

Konzepte der OOP

- OOP steht für objektorientierte Programmierung

- Konzepte:
  - Kapselung (encapsulation)
  - **Abstraktion** (abstraction)
  - Vererbung (inheritance)
  - Polymorphismus (polymorphism)

Wiederholung Abstrakte Klassen

FESTIGEN UND ERWEITERN DES WISSENS

Aufgabe

Nehmt euch nochmal 20 Minuten und schreibt euch selbst wichtige Aspekte zu abstrakten Klassen zusammen, indem ihr beispielsweise eine der folgenden Quellen nutzt:

https://www.geeksforgeeks.org/java/abstract-classes-in-java/

https://www.baeldung.com/java-abstract-class

https://medium.com/@nakulmitra2114/abstract-classes-in-java-a0bd4bb2c7d9

Was sind abstrakte Klassen? - Zusammenfassung

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Warum abstrakte Klassen?

- ermöglichen Code-Wiederverwendung
- definieren eine allgemeine Struktur
- verhindern die Instanziierung von Klassen, die nur als Basis dienen sollen

- Wie nutzen ohne Instanziierung/Objekterstellung? Wie kann man Code wiederverwenden?
  - Mittels Vererbung

Interfaces

FESTIGEN UND ERWEITERN DES WISSENS

Alltagsbeispiel

- kann wie eine Checkliste oder ein Vertrag gesehen werden
  - Steht nur drauf, was getan werden muss – nicht, wie es gemacht wird
  - Jeder, der diesen Vertrag unterschreibt, muss auch alles aus dieser Checkliste erledigen
- Beispiel:
  - Ich habe einen Vertrag unterschrieben, der sagt ich bin Dozentin
  - Im Vertrag steht, ich soll unterrichten, Unterricht vorbereiten, Unterricht evaluieren,…
  - Im Vertrag steht jedoch nicht, wie genau ich das machen soll…

Technische Grundlagen

- besondere Art von Klasse -> Schnittstelle
- 100% abstrakt
- Beinhaltet nur Signaturen von **Methoden** und **Konstanten**
- Schafft grundlegende **verpflichtende** Strukturen in Subklassen
  - **Was** soll eine Klasse können? Nicht wie
- Subklassen **müssen** alle Methoden **implementieren**
- Eine Klasse kann mehrere Interfaces implementieren
  - Sonst kann eine Klasse nicht von mehreren Klassen erben!

Beispiel

Interface
Schlüsselwort

Implements
Schlüsselwort

```
interface TeachingBehaviour{        //ANLEITUNG: WIE IST MAN EIN LEHRER?

}

class JavaTeacher implements TeachingBehaviour{

}
```

Interface statt class
Implements statt extends

Implements: Implementiert
Extends: Erweitern

Syntax der Interface-Deklaration

– Klassendefinition:

Interface TeachingBehaviour

```
interface
TeachingBehaviour { }
```

=

Interface TeachingBehaviour

```
abstract interface
TeachingBehvaviour { }
```

Was wir sehen

Was der Compiler sieht

Beispiel

```
abstract interface TeachingBehaviour{        //ANLEITUNG: WIE FLIEGT MAN?

}

class JavaTeacher implements TeachingBehaviour{

}
```

abstract ist ausgegraut, weil Interfaces per Definition abstract sind.

Inhalt eines Interfaces

- können NUR Konstanten definieren
  - Der Compiler setzt automatisch **public, static, final** vor die Konstante
- es gibt **keinen** Konstruktor!
  - Beim Versuch kommt ein Compilerfehler
- **abstrakte**, **default** und **static** – Methoden (kommen wir später dazu)

Beispiel

Konstanten!

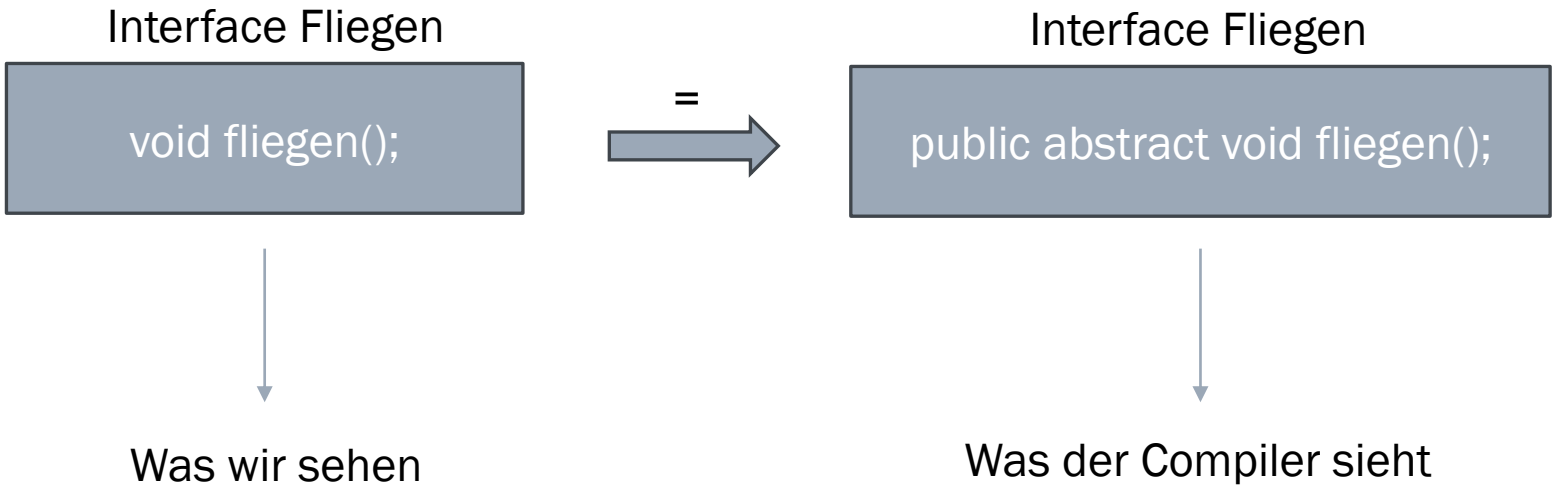
Abstrakte Methoden

```
public interface TeacherBehavior {
    int MAX_HOURS_PER_DAY = 8;

    void teach();           // Unterrichten
    void prepareLesson();   // Unterricht vorbereiten
}
```

Abstrakte Methoden

- es können abstrakte Methoden erzeugt werden
  - sind **IMMER** public
- können <u>nicht</u> final sein
- Methodendefinition:



Abstrakte Methoden

– da die Methoden abstrakt sind, brauchen sie keinen Methodenkörper

```
public abstract void test();
```

✓

```
public abstract void test(){
        System.out.println(„test“);
}
```

✗ COMPILERFEHLER!

Interface – Implementierung

- Können von **konkreten** und **abstrakten** Klassen implementiert werden
  - Schlüsselwort: **implements**
- Eine Klasse kann eine oder mehrere Interfaces implementieren

Beispiel

Interface
Schlüsselwort

Implements
Schlüsselwort

```
interface TeachingBehaviour{        //ANLEITUNG: WIE IST MAN EIN LEHRER?

}

class JavaTeacher implements TeachingBehaviour, JavaSpecificBehaviour, Calmness{

}
```

Interface statt class
Implements statt extends

Implements: Implementiert
Extends: Erweitern

Interface – Implementierung

- Ein Interface kann keine anderen Interfaces implementieren
  - Implementieren heißt: Methoden werden implementiert
  - Interface bedeutet: Methoden werden vor allem definiert

```
interface TeachingBehaviour {
    void teach();
}

interface Presentable implements TeachingBehaviour { //FEHLER
    void wearAppropriateClothes();
}
```

Interface – Implementierung

- Ein Interface kann von anderen Interfaces erben, nicht von normalen Klassen

```
interface TeachingBehaviour {
    void teach();
}

interface Presentable extends TeachingBehaviour { //Passt
    void wearAppropriateClothes();
}
```

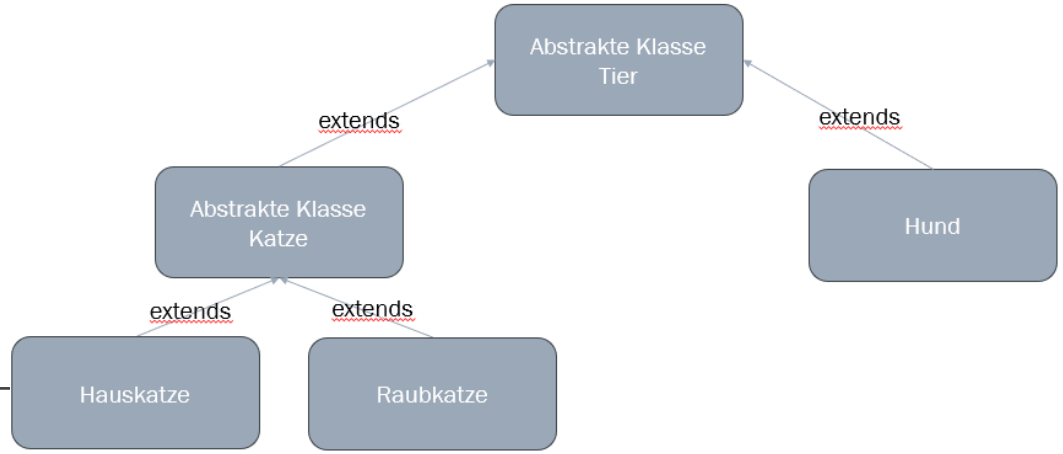
Interface – Implementierung

- Können von **konkreten** und **abstrakten** Klassen implementiert werden
  - Schlüsselwort: **implements**
- Eine Klasse kann eine oder mehrere Interfaces implementieren
- Ein Interface kann keine anderen Interfaces implementieren
- Ein Interface kann von anderen Interfaces erben
- die erste konkrete Klasse muss die noch nicht implementierten Methoden implementieren

Warum Interfaces?

- Guter „Vertrag“, der angibt, was eine Klasse an Verhaltenstypen erfüllen soll, aber nicht wie
  - Vorteil: Eine Klasse kann mehrere Interfaces implementieren, also mehrere Verträge unterschreiben
- Wiederverwendung durch Modularität und Flexibilität
  - Entkopplung ermöglicht -> Klassen sind nicht mehr so stark voneinander abhängig
- Besser wartbar

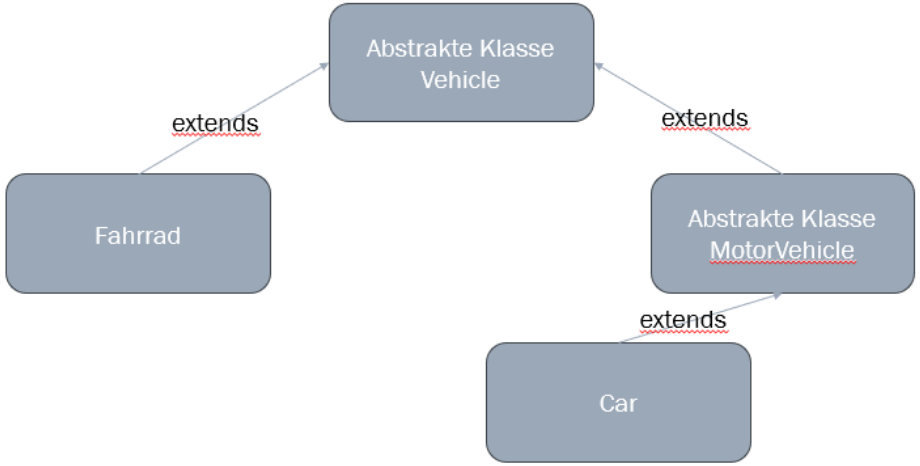
Aufgabe



Ergänze eine Klasse Vogel und eine Klasse Flugzeug

Welches Interface könnte man nun Implementieren?

Binde das Interface in deinen Code ein



Aufgabe

Recherchiert genauer zu Interfaces:

Was ist der Unterschied zwischen Klassen, Interfaces und abstrakten Klassen?

Achtet hierbei vor allem auf Komponenten, Vererbung und **Ziel.**

Abstrakte Klassen vs. Interfaces

Feature	Klassen	Abstrakte Klassen	Interface
Methoden	Ja, nur konkrete	Ja, abstrakt und konkrete	Ja, abstrakte und konkrete (behandeln wir später)
Konstruktoren	Ja	Ja	Nein
Felder	Ja	Ja	Nur Konstanten
Mehrfachvererbung	Nein, kann nur von einer Klasse erben. Kann aber mehrere Interfaces implementieren	Nein, kann nur von einer Klasse erben	Jaein, eine Klasse kann mehrere Interfaces **implementieren**
Ziel	Wiederverwenden von Methoden und Eigenschaften	Gemeinsame Basis mit teilweiser Implementierung	Verhalten definieren, ohne Implementierung vorzuschreiben

Interface – Implementierung Beispiele

- abstrakte Klassen müssen Methoden des Interfaces nicht implementieren

```
interface MyInterface {
    void myMethod();
}

abstract class MyAbstrClass implements MyInterface {
}
```

Interface – Implementierung Beispiele

- die erste konkrete Klasse jedoch schon!

```
interface MyInterface {
   void myMethod();
}

abstract class MyAbstrClass implements MyInterface {
}
```

Methoden werden überschrieben

```
public class MyClass extends MyAbstrClass {
   @Override
   public void myMethod() {
      // Implementierung der Methode
   }
}
```

Interface – Implem

```
interface MyInterface {
   void myMethod();
   void myMethod2();
}

abstract class MyAbstrClass implements MyInterface
{
   public void myMethod(){
       System.out.println(„Hello“);
   }
}
public class MyClass extends MyAbstrClass {
   public void myMethod2() {
      // Implementierung der Methode
   }
}
```

Interface – Implem

Funktioniert!

MyAbstrClass implementiert die Methode myMethod().

MyClass erbt von MyAbstrClass und muss somit nur myMethod2() implementieren.

```
interface MyInterface {
    void myMethod();
    void myMethod2();
}

abstract class MyAbstrClass implements MyInterface {
    public void myMethod(){
        System.out.println(„Hello“);
    }
}


public class MyClass extends MyAbstrClass {
    public void myMethod2() {
        // Implementierung der Methode
    }
}
```